

1. Theory Questions

1. The loss function can be described in the following way:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [1 - \log D(G(z))]$$

Where x is a real data sample, z is the noise input, and $G(z)$ is the fake data generated by the generator.

The function works by repeatedly having the generator try to fool the discriminator into classifying its false data as real data until reaching an equilibrium where the generator produces data that the discriminator cannot distinguish from real data. By using this formula, any improvement to either model will directly impact the performance of the other, forcing them to try and outplay each other and reach their optimal performance while forcing the other model to its worst performance.

2. Mode collapse is when the generator repeatedly creates duplicate data to exploit weaknesses in the discriminator and fails to create diverse, realistic data. If the generator notices certain attributes in its generated data improve its performance against the discriminator, it will be rewarded for continuing this pattern of exploitation. If the discriminator fails to recognize this pattern, it will continue to be exploited until an apparent optimum performance is reached. It can be mitigated using a variety of methods including batch normalization, minibatch discrimination, or WGANs.

3. The role of the discriminator is to classify data as either real (from the dataset) or fake (generated by the generator). It is trained to continually improve its ability to distinguish real from fake data. As the discriminator becomes more accurate, the generator must improve in order to produce data that can successfully fool the discriminator. This makes the two directly adversarial, driving both models to improve over time.

4. Inception Score (IS) works by evaluating the diversity and confidence of classification of data created by the generator. Higher scores mean the generated images are both recognizable and varied across classes, showing the generator is producing high-quality, diverse outputs.

Frechet Inception Distance (FID) can be used to measure the distance between the set of generated images and the real data. Lower FID values indicate that the generator's outputs are closer in distribution to real data, capturing both realism and diversity.

2. GAN

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
import os

# Load and preprocess data
(train_images, _), (_, _) = tf.keras.datasets.cifar10.load_data()
train_images = train_images.astype('float32')
train_images = (train_images - 127.5) / 127.5

BUFFER_SIZE = 50000
BATCH_SIZE = 256

train_dataset = tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

# Generator model
def make_generator_model():
    model = tf.keras.Sequential([
        layers.Dense(8*8*256, use_bias=False, input_shape=(100,)),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Reshape((8, 8, 256)),

        layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding='same', use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
```

```

layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same', use_bias=False),
layers.BatchNormalization(),
layers.LeakyReLU(),

#Extra Convolutional Layers
layers.Conv2DTranspose(32, (5, 5), strides=(1, 1), padding='same', use_bias=False),
layers.BatchNormalization(),
layers.LeakyReLU(),

layers.Conv2DTranspose(16, (3, 3), strides=(1, 1), padding='same', use_bias=False),
layers.BatchNormalization(),
layers.LeakyReLU(),

    layers.Conv2DTranspose(3, (5, 5), strides=(2, 2), padding='same', use_bias=False, activation='tanh')
])
return model

# Discriminator model
def make_discriminator_model():
    model = tf.keras.Sequential([
        layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same', input_shape=[32, 32, 3]),
        layers.LeakyReLU(),
        layers.Dropout(0.3),

        layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'),
        layers.LeakyReLU(),
        layers.Dropout(0.3),

        layers.Flatten(),
        layers.Dense(1)
    ])
    return model

# Loss functions
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

# Optimizers
generator = make_generator_model()
discriminator = make_discriminator_model()
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

# Training function
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, 100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))

#Image saving after every 10 Epochs

# Create a folder to save images
os.makedirs('generated_images', exist_ok=True)

# Function to generate and save images
def generate_and_save_images(model, epoch, num_images=16):

```

```

def generate_and_save_images(model, epoch, num_images=10):

    noise = tf.random.normal([num_images, 100])
    predictions = model(noise, training=False)

    fig = plt.figure(figsize=(4,4))
    for i in range(num_images):
        plt.subplot(4, 4, i+1)
        plt.imshow((predictions[i] + 1) / 2)
        plt.axis('off')

    plt.savefig(f'image_at_epoch_{epoch:03d}.png')
    plt.close(fig)

# Training loop
def train(dataset, epochs):
    for epoch in range(epochs):
        for image_batch in dataset:
            train_step(image_batch)
            print(f'Epoch {epoch+1} completed')

        if (epoch + 1) % 10 == 0:
            generate_and_save_images(generator, epoch + 1)

# Run the training
train(train_dataset, epochs=100)

```

```

Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Epoch 6 completed
Epoch 7 completed
Epoch 8 completed
Epoch 9 completed
Epoch 10 completed
Epoch 11 completed
Epoch 12 completed
Epoch 13 completed
Epoch 14 completed
Epoch 15 completed
Epoch 16 completed
Epoch 17 completed
Epoch 18 completed
Epoch 19 completed
Epoch 20 completed
Epoch 21 completed
Epoch 22 completed
Epoch 23 completed
Epoch 24 completed
Epoch 25 completed
Epoch 26 completed
Epoch 27 completed
Epoch 28 completed
Epoch 29 completed
Epoch 30 completed
Epoch 31 completed
Epoch 32 completed
Epoch 33 completed
Epoch 34 completed
Epoch 35 completed
Epoch 36 completed
Epoch 37 completed
Epoch 38 completed
Epoch 39 completed
Epoch 40 completed
Epoch 41 completed
Epoch 42 completed
Epoch 43 completed
Epoch 44 completed
Epoch 45 completed
Epoch 46 completed
Epoch 47 completed
Epoch 48 completed
Epoch 49 completed
Epoch 50 completed
Epoch 51 completed
Epoch 52 completed
Epoch 53 completed

```

```
Epoch 54 completed  
Epoch 55 completed  
Epoch 56 completed  
Epoch 57 completed  
Epoch 58 completed
```